

2026

Cyanea

Universidad Tecnológica Nacional



- Chialva Fátima
- Correa Luciano
- Gatica Andrea Ticiana
- Giampieri Lucia
- Paez Maria Candela

 Universidad Tecnológica Nacional		FACULTAD REGIONAL CÓRDOBA Ingeniería en Sistemas de Información Proyecto Final	
Cátedra	Proyecto Final		
Ámbito de aplicación	CYANEA		
Nombre del Software	CYANEA		
Objetivo	-		
Metodología	Scrum		
Equipo de Trabajo	Legajo	Apellido, Nombres	Mail de Contacto
	95147	Chialva, Fatima	fatimachialva@gmail.com
	69323	Correa, Luciano Joaquín	correalucianojoaquin@gmail.com
	94371	Gatica, Andrea Ticiania	ticii.gatica@gmail.com
	96505	Giampieri, Lucia Belén	luciagiampieri24@gmail.com
	95256	Paez, María Candela	candelapaez62@gmail.com
Curso	5K4		
Docentes	Silvina Arenas		
Año de Cursado	2026		

Tabla de contenidos

Introducción.....	4
Propósito del documento.....	4
Alcance.....	4
Audiencia.....	4
Propuesta Tecnológica del Sistema.....	5
1. Arquitectura del sistema.....	5
1.1 Arquitectura del Cliente.....	5
1.2 Arquitectura del Servidor.....	6
2. Stack tecnológico.....	6
2.1 Justificación del stack tecnológico.....	7
Frontend Mobile – React Native (TypeScript).....	7
Backend – FastAPI (Python).....	8
Base de Datos Principal – PostgreSQL.....	8
Acceso a Datos (ORM) – SQLAlchemy.....	8
Migraciones de Base de Datos – Alembic.....	9
Base de Datos Local – SQLite.....	9
Autenticación – JWT + OAuth.....	9
Geolocalización y Mapas – Google Maps Platform.....	9
Notificaciones Push – Firebase Cloud Messaging (FCM).....	10
Almacenamiento de Archivos – Supabase Storage.....	10
Historial de Revisión.....	11

Introducción

Propósito del documento

El presente documento tiene como propósito definir y justificar la arquitectura de software y el conjunto de tecnologías (Stack) seleccionadas para el desarrollo de la plataforma CYANEA. La selección se ha basado en criterios de escalabilidad, mantenibilidad, velocidad de desarrollo y calidad de la documentación generada.

Alcance

El alcance de este documento abarca la definición, selección y justificación técnica del stack tecnológico y la arquitectura de software propuesta para el proyecto CYANEA. Esto incluye la estrategia de desarrollo basada en el paradigma de Progressive Web App (PWA), la selección de lenguaje y la arquitectura de persistencia de datos. No incluye el desarrollo de código fuente final de las funcionalidades del producto, la configuración de entornos de despliegue en producción, ni la gestión operativa de servicios externos contratados más allá de su integración técnica inicial.

Audiencia

Este documento está dirigido a las siguientes personas y roles:

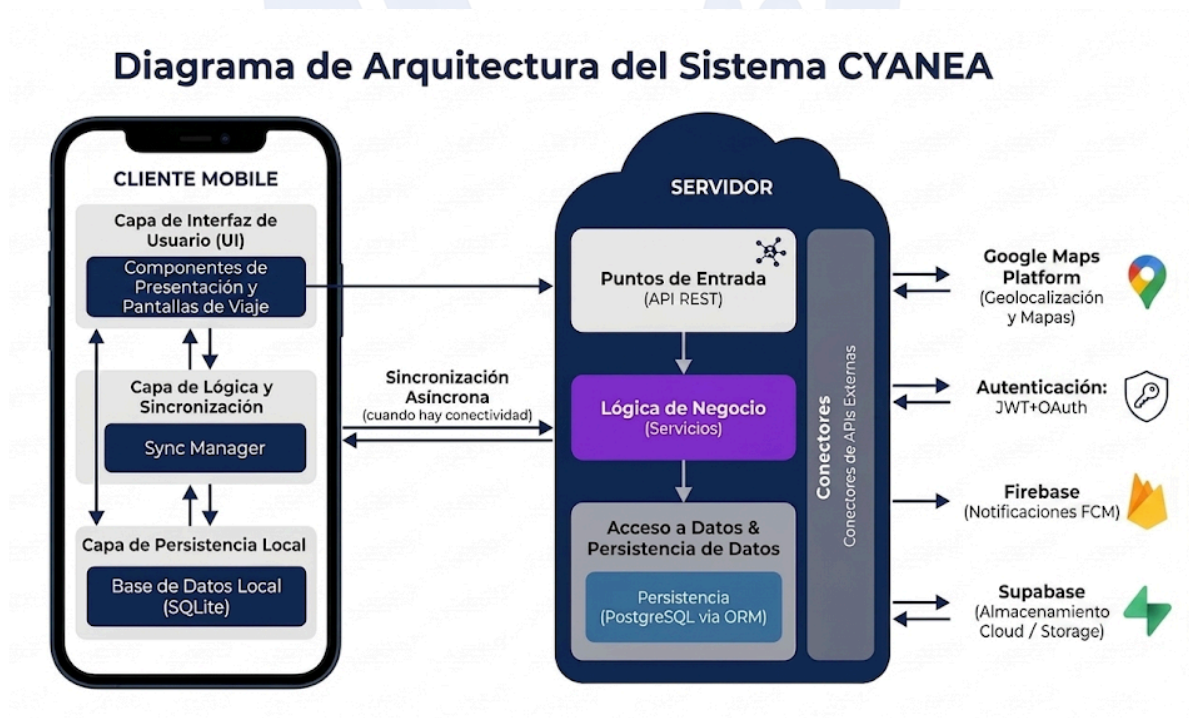
Audiencia	Rol	Conocimiento previo asumido
Directora de Tesis	Silvina Arenas	Conocimiento académico en IS; supervisa coherencia metodológica.
Equipo de desarrollo	Autores del proyecto	Conocimiento técnico en desarrollo de software; familiaridad con el dominio del proyecto.

Propuesta Tecnológica del Sistema

1. Arquitectura del sistema

El sistema se basa en una arquitectura cliente–servidor desacoplada, donde el backend se organiza mediante un patrón de arquitectura en capas. Este enfoque permite distribuir las responsabilidades en componentes lógicos independientes, con el objetivo de mejorar la mantenibilidad, escalabilidad y claridad del sistema.

Por su parte, la aplicación móvil (cliente) implementa una estrategia de operación offline parcial, que permite el uso sin conexión a internet de funcionalidades críticas como la gestión local de itinerarios, checklists y registros de gastos. La información generada localmente se almacena en el dispositivo y se sincroniza posteriormente de forma asíncrona cuando el sistema detecta la recuperación de la conectividad.



1.1 Arquitectura del Cliente

Capa	Componente	Descripción
Capa de Interfaz de Usuario (UI)	UI Mobile	Presenta las pantallas e interfaces de la aplicación y gestiona la interacción directa con el usuario.
Capa de Lógica y Sincronización	Sync Manager	Gestiona la lógica interna del dispositivo y coordina la sincronización asíncrona de los

		datos locales con el backend cuando hay conectividad.
Capa de Persistencia Local	Base de Datos Local	Almacena información del sistema en el dispositivo, permitiendo el funcionamiento offline y acceso de baja latencia a los datos.

1.2 Arquitectura del Servidor

Capa	Componente	Descripción
Capa de Entrada	Controladores	Expone la API REST del sistema, recibe solicitudes HTTP, valida los datos de entrada y delega la ejecución a la capa de servicios.
Capa de Negocio	Servicios	Contiene la lógica del sistema, incluyendo gestión de usuarios, viajes colaborativos, gastos, itinerarios y votaciones.
Capa de Persistencia	Modelos y Repositorios ORM	Gestiona el mapeo objeto-relacional y el acceso a la base de datos principal del sistema.
Capa de Integración Externa	Conectores de APIs	Maneja la comunicación con servicios externos como mapas, autenticación, almacenamiento en la nube y notificaciones.

2. Stack tecnológico

Capa/Módulo	Tecnología	Lenguaje	Rol en el sistema
Frontend Mobile	React Native (con expo)	Typescript	Desarrollo de la aplicación móvil multiplataforma (Android/iOS)
Backend	FastAPI	Python	Desarrollo de la API REST y lógica de negocio
Base de Datos Principal	PostgreSQL	SQL	Persistencia central de datos relacionales del sistema
Acceso a Datos	SQLAlchemy	Python	Mapeo objeto-relacional

(ORM)			entre el backend y la base de datos
Migraciones de Base de Datos	Alembic	Python	Gestión y versionado de cambios en el esquema de la base de datos
Base de Datos Local (Offline)	SQLite	SQL	Almacenamiento local de datos para funcionamiento sin conexión
Autenticación	JWR+OAuth	-	Gestión de sesiones, autenticación segura e inicio de sesión con terceros
Geolocalización y Mapas	Google Maps Platform	-	Servicios de mapas, rutas, geolocalización y puntos de interés
Notificaciones Push	Firebase Cloud Messaging (FCM)	-	Envío de notificaciones en tiempo real a dispositivos móviles
Almacenamiento de Archivos	Supabase Storage	-	Gestión de documentos, imágenes y archivos asociados a viajes

2.1 Justificación del stack tecnológico

Frontend Mobile – React Native (TypeScript)

Se selecciona React Native por permitir el desarrollo de una única aplicación multiplataforma para Android e iOS, reduciendo tiempos de desarrollo y mantenimiento al compartir una sola base de código.

El uso de Expo simplifica la configuración del entorno, acelera el desarrollo y facilita el acceso a funcionalidades nativas del dispositivo como geolocalización, almacenamiento local y notificaciones push, sin necesidad de configuraciones nativas complejas.

La integración con TypeScript aporta tipado estático, mayor robustez y mantenibilidad del código, reduciendo errores en tiempo de desarrollo.

Este stack permite implementar de forma eficiente las funcionalidades centrales del sistema y deja abierta la posibilidad de una futura extensión hacia una plataforma web dentro del mismo ecosistema tecnológico.

Backend – FastAPI (Python)

Para el desarrollo del backend se seleccionó el framework FastAPI debido a su alto rendimiento operativo y su eficiencia en la construcción de APIs REST modernas. Esta tecnología permite implementar de manera óptima la lógica de negocio central del sistema —incluyendo la gestión de usuarios, viajes colaborativos, gastos, balances y documentación—, además de proporcionar el soporte necesario para la sincronización asincrónica de datos entre los dispositivos móviles y el servidor.

La adopción de Python como lenguaje principal simplifica el desarrollo y mantenimiento de la plataforma, ofreciendo una gran flexibilidad arquitectónica para futuras ampliaciones modulares. Asimismo, FastAPI genera documentación automática e interactiva de la API, lo que agiliza significativamente las etapas de prueba, depuración e integración cruzada con el frontend.

Base de Datos Principal – PostgreSQL

Para la gestión de la base de datos del sistema se selecciona PostgreSQL debido a su solidez como sistema relacional y su capacidad para modelar estructuras de datos complejas y altamente interconectadas.

El sistema requiere la representación de entidades fuertemente relacionadas, tales como usuarios, viajes, participantes, gastos, votaciones, itinerarios, checklists y documentos, lo que hace necesario el uso de una base de datos robusta y consistente.

PostgreSQL garantiza la integridad de los datos mediante el uso de transacciones, ofreciendo además un alto rendimiento en operaciones concurrentes y una arquitectura escalable para futuras ampliaciones del sistema. Asimismo, permite la ejecución de consultas complejas, lo que resulta fundamental para el procesamiento y análisis de la información generada dentro de la plataforma.

Acceso a Datos (ORM) – SQLAlchemy

Para la gestión del acceso a datos se utiliza SQLAlchemy como herramienta de Object-Relational Mapping (ORM), permitiendo representar las entidades del sistema como objetos dentro del lenguaje Python.

Esta aproximación simplifica la interacción con la base de datos, reduciendo la necesidad de escribir consultas SQL manuales y mejorando la legibilidad y mantenibilidad del código. Además, favorece la evolución del sistema al desacoplar la lógica de negocio de la capa

de persistencia, facilitando modificaciones futuras en la estructura de datos sin impactar directamente en la implementación del backend.

Migraciones de Base de Datos – Alembic

Para la gestión de cambios en la estructura de la base de datos se incorpora Alembic como herramienta de migraciones y versionado del esquema.

Cada modificación en la estructura de la base de datos se registra como una migración, lo que permite mantener la consistencia entre los distintos entornos del sistema, incluyendo desarrollo, pruebas y producción.

Este enfoque facilita el trabajo colaborativo en equipo y evita desincronizaciones entre el código y la base de datos, garantizando un control adecuado sobre la evolución del modelo de datos a lo largo del tiempo.

Base de Datos Local – SQLite

Para la gestión de datos en modo offline se utiliza SQLite como base de datos local embebida en la aplicación móvil.

Esta tecnología permite el almacenamiento de información crítica del sistema, como gastos, itinerarios, checklists y documentos, asegurando la continuidad del funcionamiento de la aplicación sin conexión a internet. Posteriormente, los datos almacenados localmente pueden sincronizarse con el servidor cuando se restablece la conectividad, garantizando la consistencia de la información entre el dispositivo y la base de datos central.

Autenticación – JWT + OAuth

Para la gestión de autenticación y control de acceso se utiliza JWT (JSON Web Token), permitiendo la identificación segura de los usuarios y la protección de los endpoints de la API mediante tokens de acceso.

Asimismo, se incorpora OAuth para habilitar el inicio de sesión mediante proveedores externos como Google y Facebook, mejorando la experiencia de usuario y simplificando el proceso de autenticación dentro del sistema.

Geolocalización y Mapas – Google Maps Platform

Para la implementación de funcionalidades de geolocalización y visualización de mapas se utiliza Google Maps Platform, permitiendo la exploración de destinos, búsqueda de puntos de interés, generación de rutas y seguimiento de recorridos dentro del sistema.

Esta tecnología se selecciona por ser una solución ampliamente adoptada, estable y con una extensa documentación, lo que facilita su integración en aplicaciones móviles y garantiza una experiencia de usuario consistente y confiable.

Notificaciones Push – Firebase Cloud Messaging (FCM)

Para el envío de notificaciones push en tiempo real se utiliza Firebase Cloud Messaging (FCM), permitiendo la comunicación inmediata de eventos relevantes dentro del sistema. Esta herramienta se emplea para notificar recordatorios de actividades, alertas de gastos, invitaciones a viajes y actualizaciones en itinerarios o votaciones, mejorando la interacción y la experiencia del usuario.

Su integración con React Native y su alta escalabilidad la convierten en una solución adecuada para soportar el crecimiento futuro del sistema y el aumento en la cantidad de usuarios y eventos.

Almacenamiento de Archivos – Supabase Storage

Para el almacenamiento de archivos del sistema se utiliza Supabase Storage, permitiendo la gestión de documentos asociados a los viajes, tales como imágenes, archivos PDF y otros recursos relevantes. Esta solución proporciona almacenamiento seguro, control de permisos y generación de URLs de acceso, asegurando que solo los usuarios autorizados puedan visualizar o descargar la información.

Asimismo, ofrece escalabilidad para acompañar el crecimiento del sistema e integración sencilla con la aplicación móvil, facilitando la carga y recuperación de archivos dentro de la plataforma.

Historial de Revisión

La siguiente tabla registra todos los cambios realizados sobre este documento. Cada nueva versión debe incrementar el número de versión, consignar la fecha del cambio, indicar qué integrante realizó la modificación y describir brevemente en qué consistió.

Versión	Fecha	Autores	Descripción del cambio
1.0	28/05/2026	Chialva, Fátima Gatica, A. Ticiania Giampieri, Lucía B. Paez, M.Candela Correa, Luciano J.	Creación inicial del documento.